Swarm Agent SDK

Hook System, Task Enforcement, and Deterministic Behavior Architecture

Version 1.0 — April 2026

Swarm Engineering

Abstract

This document describes the complete architecture of the Swarm Agent SDK's hook system, task management, steering agent, and data pipeline. It details how these systems compose to produce deterministic, observable agent behavior: every tool call is intercepted, evaluated for task relevance, gated by enforcement rules, and recorded through a three-tier data pipeline (Bronze → Silver → Gold). The document includes R-generated visualizations and TikZ flow diagrams.

Contents

1 System Overview **3**

2 Hook System Architecture **3**
2.1 Execution Model . . . 3
2.2 Priority Landscape . . . 5
2.3 Hook–Event Subscription Matrix . . . 6
2.4 Complete Hook Registry . . . 6

3 Task Management System **7**
3.1 Task State Machine . . . 7
3.2 TodoItem Structure . . . 7
3.3 Task Categories and Enforcement . . . 8
3.4 Task Enforcement Flow . . . 9
3.5 Task Guidance Post-Hook . . . 9

4 Steering Agent **9**
4.1 Architecture . . . 10
4.2 Decision Types . . . 10
4.3 Relevance Context . . . 10

5 Plan Mode Integration **10**

6 Data Pipeline: Bronze → Silver → Gold **12**
6.1 Three-Tier Architecture . . . 12
6.2 Bronze Tier: Raw Capture . . . 12
6.3 Silver Tier: LLM-Scored Findings . . . 12

6.3.1 Analyzer Chain . . . 13
6.3.2 Batch LLM Scoring . . . 13
6.4 Gold Tier: Persistent Memory . . . 13
6.4.1 Consolidation Trigger . . . 13

7 Deterministic Behavior Guarantees 14
7.1 Invariant 1: No Tool Without Task . . . 14
7.2 Invariant 2: Category-Based Tool Gating . . . 14
7.3 Invariant 3: Priority-Ordered Execution . . . 14
7.4 Invariant 4: Guidance After Task Operations . . . 14
7.5 Invariant 5: Complete Event Capture . . . 14

8 Complete Execution Trace 14

9 Dependency Graph 15

10 Summary 16

1 System Overview

The Swarm SDK enforces a deterministic agent workflow through four interlocking subsystems:

1. **Hook System** — Priority-ordered interceptors that wrap every event (tool calls, messages, provider requests, lifecycle transitions).
2. **Task Management** — A mandatory task tracker that gates tool execution; no task = no tool.
3. **Steering Agent** — An LLM-backed evaluator (Ring 2) that assesses tool relevance against the active task.
4. **Data Pipeline** — A Bronze → Silver → Gold funnel that captures, scores, and consolidates agent activity into persistent memory.

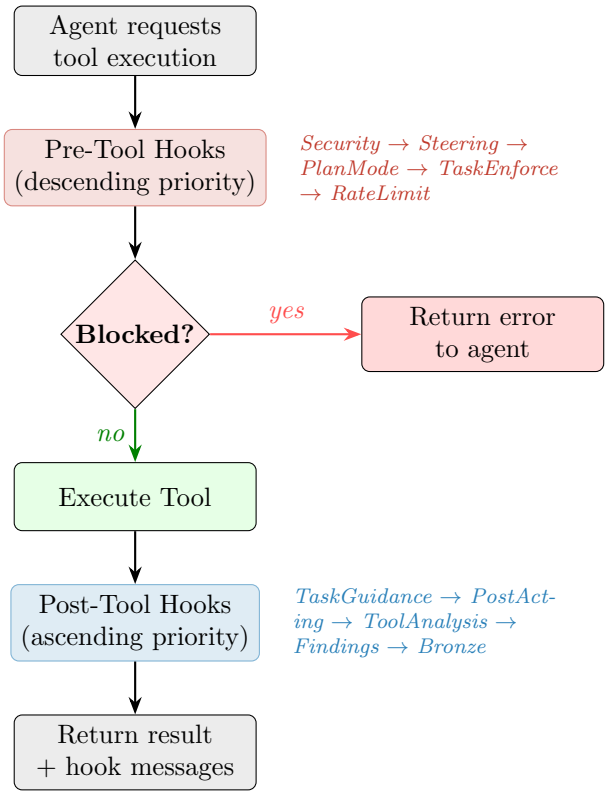

Figure 1: Core tool execution flow showing hook interception points.

2 Hook System Architecture

2.1 Execution Model

All hooks implement the `Hook` interface:

```
type Hook interface {
    OnEvent(ctx context.Context, event Event) (HookResult, error)
    Filter(event Event) bool
```

```
    Priority() int
    Name() string
}
```

The `Manager` collects all applicable hooks (global + scoped), sorts them by priority **descending**, and executes them sequentially. The three possible actions are:

Action	Constant	Effect
Continue	`ActionContinue`	Pass event to next hook
Block	`ActionBlock`	Stop chain; return error
Modify	`ActionModify`	Replace event; continue

When any hook returns `ActionBlock`, execution halts immediately. The blocked message is returned to the agent as an error. No subsequent hooks run, and the tool is never executed.

2.2 Priority Landscape

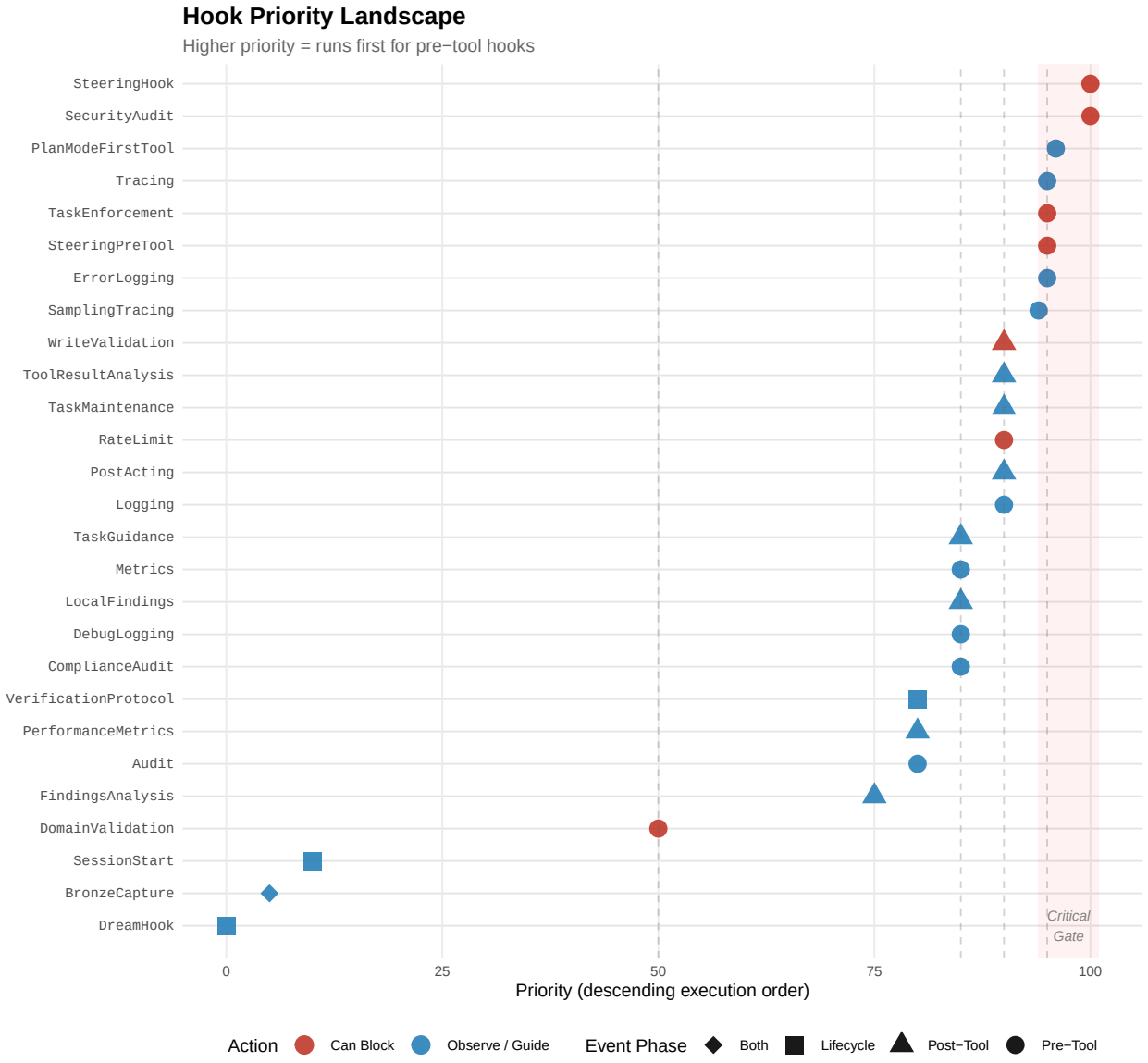

Figure 2: All registered hooks plotted by priority. Red dots can block execution; blue dots are observational. Shape indicates the event phase.

2.3 Hook–Event Subscription Matrix

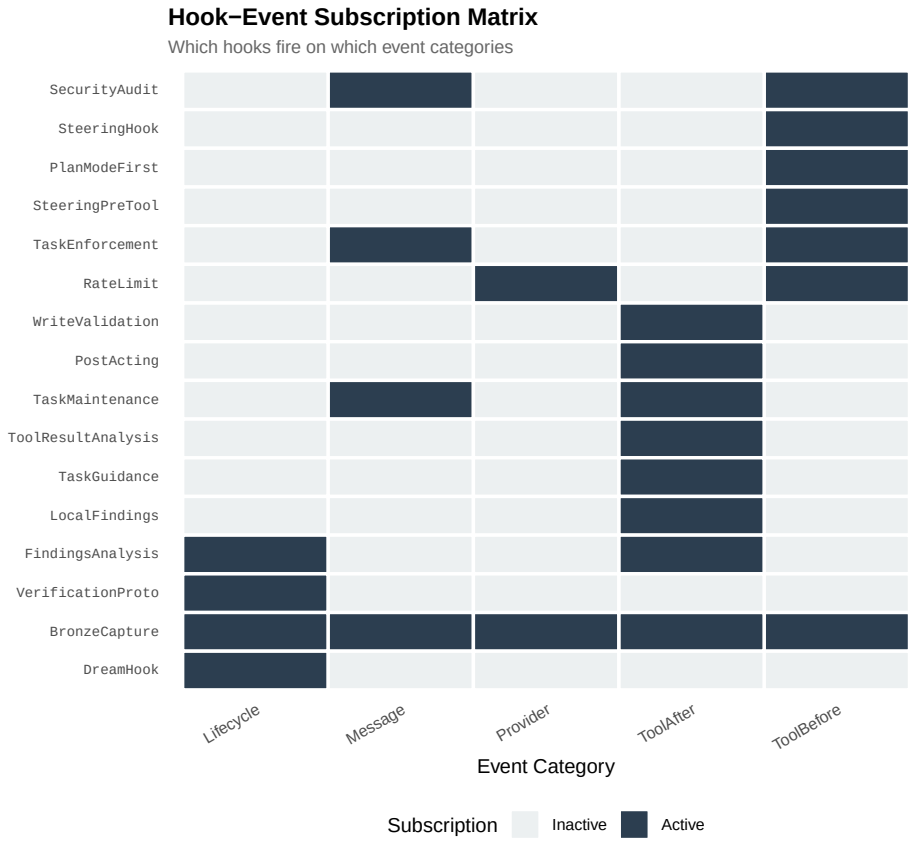

Figure 3: Which hooks subscribe to which event categories. BronzeCapture subscribes to all events; most hooks specialize on tool events.

2.4 Complete Hook Registry

Hook	Pri.	Block?	Purpose
SecurityAuditHook	100		PII detection / redaction
SteeringHook	100		LLM tool-call approval gate
PlanModeFirstToolHook	96		Inject problem decomposition prompt
SteeringPreToolHook	95		Task-relevance + category enforcement
TaskEnforcementHook	95		Require tasks before any tool
TracingHook	95		Distributed tracing spans
ErrorLoggingHook	95		Log error events
SamplingTracingHook	94		Sampled tracing (configurable rate)
RateLimitHook	90		Token-bucket rate limiting
WriteValidationHook	90		Block repeated Write failures
PostActingHook	90		Remind to verify/document after acting
TaskMaintenanceHook	90		Periodic task health reminders
ToolResultAnalysisHook	90		Run 5 analyzers, tag findings

Hook	Pri.	Block?	Purpose
LoggingHook	90		General event logging
TaskGuidanceHook	85		Guide after TaskCreate/Update
LocalFindingsHook	85		Persist findings to file cache
DebugLoggingHook	85		Verbose debug logging
ComplianceAuditHook	85		Full event audit trail
MetricsHook	85		Counter/histogram metrics
AuditHook	80		Sensitive event audit trail
VerificationProtocolHook	80		End-of-session verification
PerformanceMetricsHook	80		Latency histograms
FindingsAnalysisHook	75		LLM batch scoring of findings
DomainValidationHook	50		Custom business rule enforcement
SessionStartHook	10		Session workflow guidance
BronzeEventHook	5		Capture all events to JSONL
DreamHook	0		Memory consolidation trigger

3 Task Management System

3.1 Task State Machine

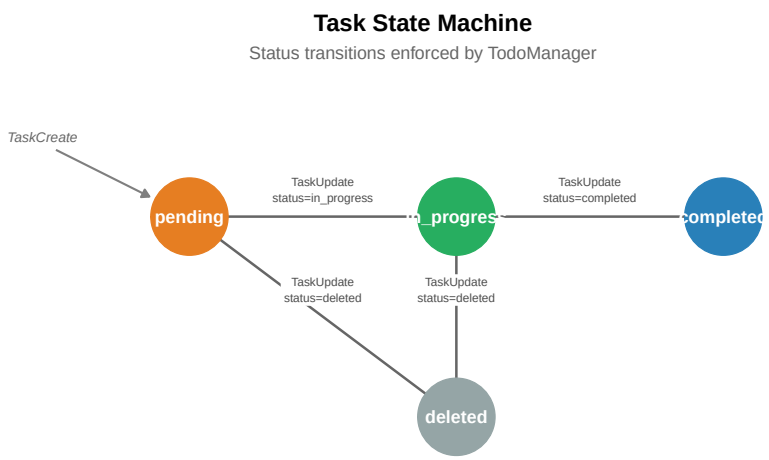

Figure 4: Task lifecycle states and valid transitions. All tasks begin as `pending` via `TaskCreate`.

3.2 TodoItem Structure

Each task is represented by a `TodoItem` in the global `TodoManager` singleton:

```go
type TodoItem struct {
    ID         string       // Auto-generated sequential ID
    Content    string       // Imperative subject ("Fix auth bug")
    Status     TodoStatus   // pending | in_progress | completed
    Priority   TodoPriority // low | medium | high
    Category   TaskCategory // researching | planning | acting | ...
    Notes      []string     // Timestamped observation entries
```

```
    DependsOn  []string     // Tasks that must complete first
    Blocks     []string     // Tasks waiting on this one (computed)
    OwnerID    string       // Agent/subagent owner
    Sequence   int          // Display order within owner group
}
```

3.3 Task Categories and Enforcement

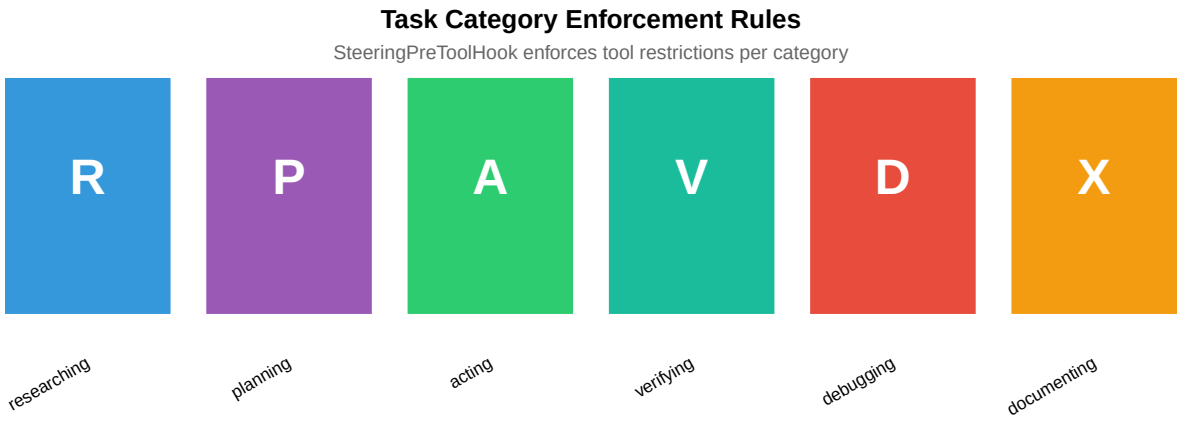

Figure 5: Six task categories with their steering enforcement rules. Categories `planning` and `debugging` trigger hard blocks on write/execute tools.

The `SteeringPreToolHook` reads the active task's category and applies deterministic rules *before* any LLM evaluation:

Hard Block Rules

- **Planning task**: Write, Edit, Bash, Shell are blocked. Only read-only research tools are permitted.
- **Debugging task**: Only diagnostic tools (Read, Grep, Glob, git log, git diff) are allowed. All other tools are blocked.

3.4 Task Enforcement Flow

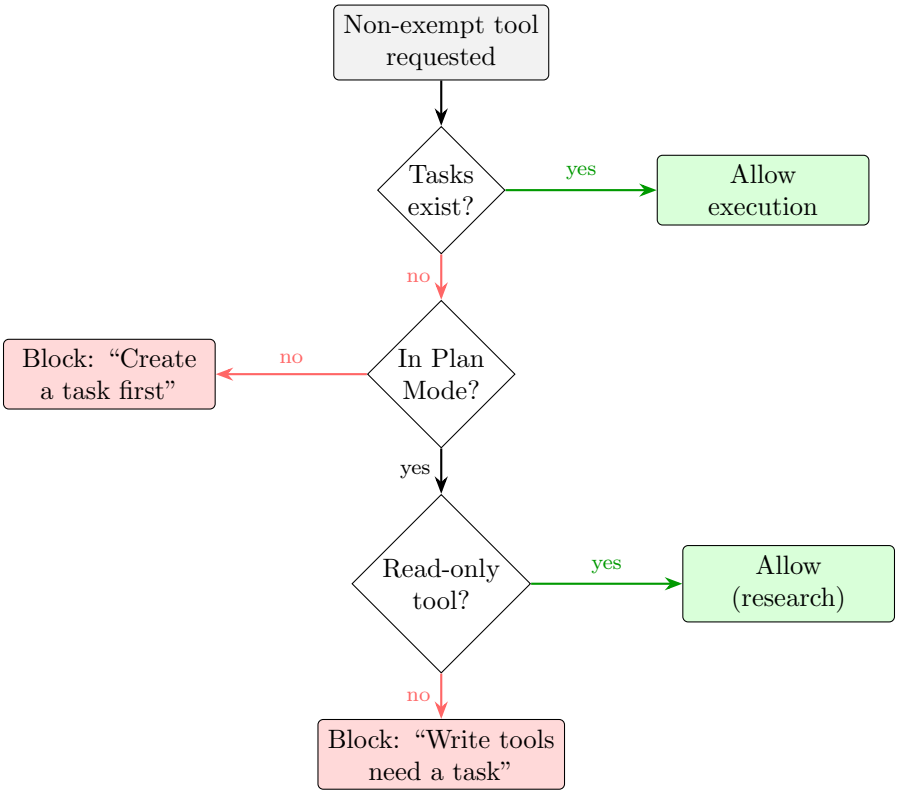

Figure 6: TaskEnforcementHook decision tree. Plan mode enables read-only research without requiring tasks; write tools always require a task.

3.5 Task Guidance Post-Hook

The `TaskGuidanceHook` (priority 85, post-tool) fires after task operations and injects contextual guidance:

After Operation	Guidance Injected
`TaskCreate` (no active task)	"Use TaskUpdate status=in_progress"
`TaskUpdate` → `in_progress`	"Task is ACTIVE, proceed"
`TaskUpdate` → `completed`	"Pick up next pending task"

This prevents wasted tokens from the agent discovering constraints through trial and error.

4 Steering Agent

The `SteeringAgent` is a Ring 2 oversight mechanism that uses an LLM to evaluate whether tool calls are relevant to the active task.

4.1 Architecture

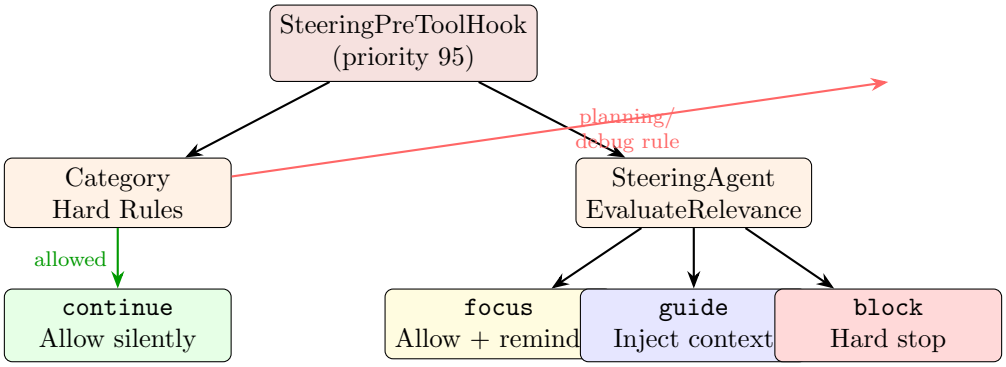

Figure 7: Steering evaluation flow. Deterministic category rules run first; LLM evaluation only fires when the category check passes.

4.2 Decision Types

Decision	Behavior
`continue`	Tool is on-task. Allow silently with no message injection.
`focus`	Tool is drifting. Allow execution but inject a one-line reminder about the current task objective.
`guide`	Tool relates to known project context. Allow and inject relevant dream memories or project references.
`block`	Tool is off-topic or contradicts the plan. Hard stop with explanation and suggested alternative.

4.3 Relevance Context

The `RelevanceContext` sent to the LLM includes:

- Tool name and parameters
- Active task title, category, description
- Task history (completed tasks)
- Active plan text
- Recent conversation messages
- Dream memories (long-term)
- Tools used in current phase
- Artifacts produced so far

5 Plan Mode Integration

Plan mode and task enforcement work together through a two-hook collaboration:

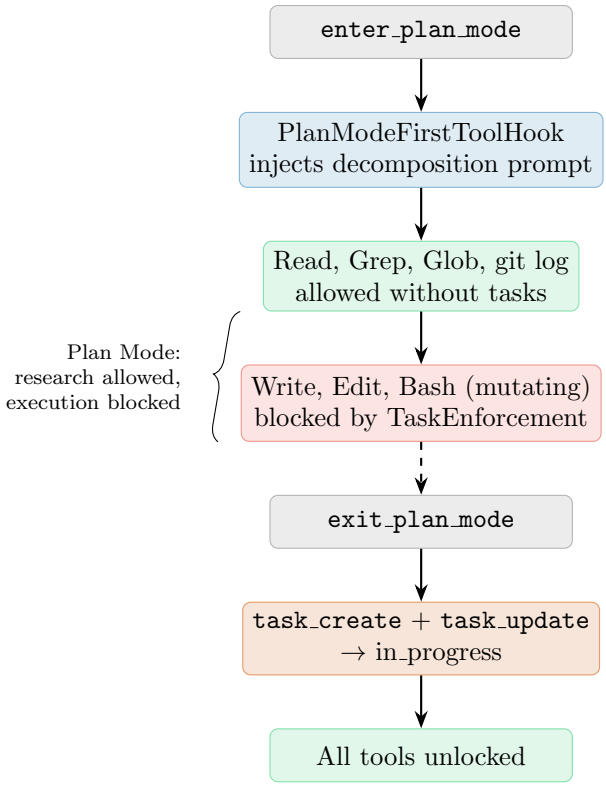

Figure 8: Plan mode workflow. Research tools are unlocked immediately; write tools require exiting plan mode and creating tasks.

6 Data Pipeline: Bronze → Silver → Gold

6.1 Three-Tier Architecture

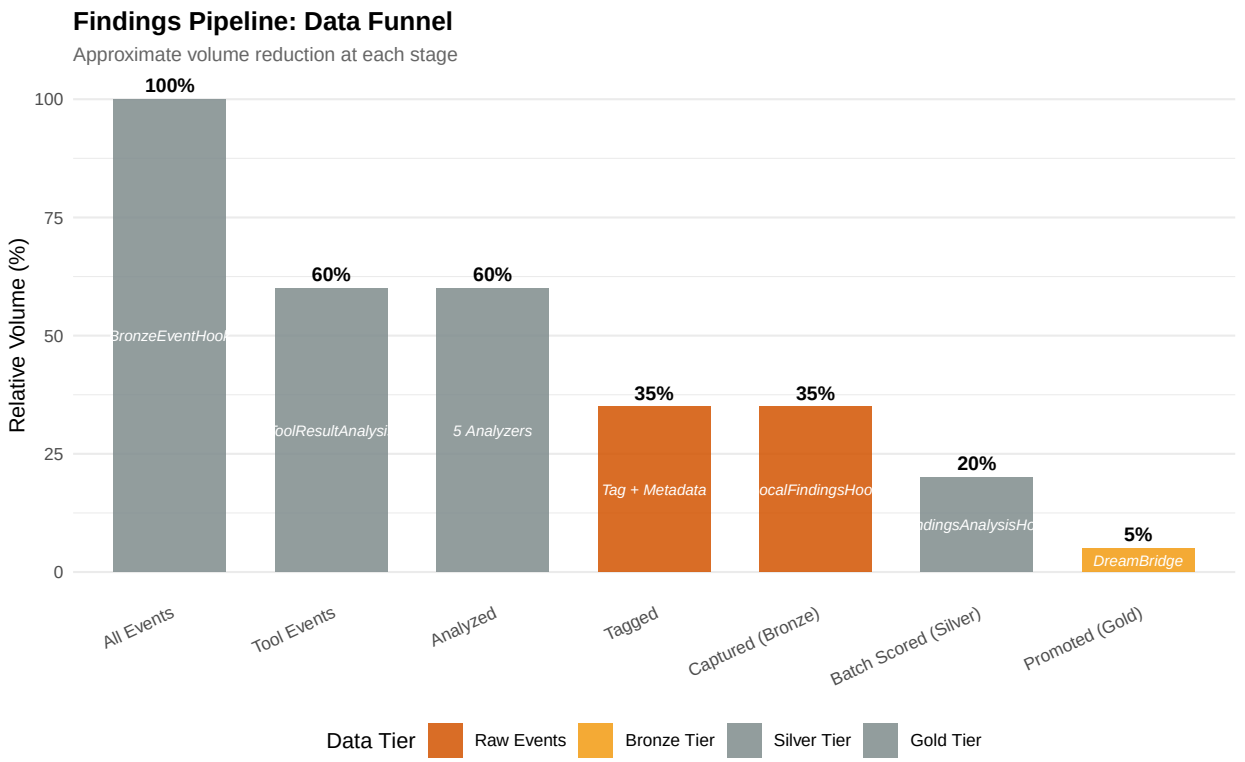

Figure 9: Data volume reduction through the pipeline. All events enter at Bronze; only high-value findings are promoted to Gold (persistent memory).

6.2 Bronze Tier: Raw Capture

Hook: `BronzeEventHook` (priority 5, all events)

Every event is written as a JSON line to append-only, date-partitioned files:

```
~/.swarm/projects/<hash>/bronze/2026/04/16/events_14.jsonl
```

Each bronze event includes:

- Timestamp, event type, agent ID, conversation ID
- **Active task ID** (resolved from `TodoManager` at capture time)
- Summarized tool input/output (truncated to 200 chars)
- Full raw event data

The task ID correlation enables querying all bronze events associated with a specific task, effectively turning the bronze layer into a task-linked artifact store.

6.3 Silver Tier: LLM-Scored Findings

Hooks: `ToolResultAnalysisHook` (tagging) → `LocalFindingsHook` (capture) → `FindingsAnalysisHook` (scoring)

6.3.1 Analyzer Chain

Five analyzers run on every post-tool event, their results merged by union:

Analyzer	What It Detects
DefaultAnalyzer	Captures evaluate/test/build/deploy/benchmark tools; failures bumped to priority 70
ErrorAnalyzer	Any error/stderr/failure → priority 90, creates task
InsightAnalyzer	Metric keywords (accuracy, f1, coverage) → tagged
DreamAnalyzer	Failure patterns, recipes, file relationships, semantic insights → tagged for memory promotion
SteeringAnalyzer	Inefficiency (`grep -r`), context pressure (>5000 tokens), batch opportunities → tagged for guidance

6.3.2 Batch LLM Scoring

`FindingsAnalysisHook` accumulates findings and periodically sends batches (up to 20) to `SteeringAgent.Evaluat`
The LLM returns:

```go
type FindingsEvaluation struct {
    FindingID     string  // Finding to score
    Priority      float64 // 0.00-100.00 (2 decimal places)
    ShouldPromote bool    // true when Priority >= 70.00
    Reasoning     string  // LLM's justification
    Tags          []string
}
```

Findings with `ShouldPromote = true` are sent to the Dream bridge.

6.4 Gold Tier: Persistent Memory

Hooks: `FindingsToDreamBridge` (promotion) → `DreamHook` (consolidation)

The Dream system converts high-value findings into structured memory entries:

Finding Tag	Memory Type
`anti-pattern`	`feedback` (what to avoid)
`recipe`	`project` (reusable patterns)
`relationship`	`reference` (file/system links)
`insight`	`feedback` (learned behaviors)

6.4.1 Consolidation Trigger

The `DreamHook` fires on `EventConversationCompleted` and checks:

$$\text{ShouldRun} = \text{Enabled} \wedge (\Delta t_{\text{scan}} \geq 10\text{min}) \wedge (\Delta t_{\text{last}} \geq 24\text{h}) \wedge (n_{\text{sessions}} \geq 5)$$

When all conditions are met, a restricted `BackgroundAgent` (read/write filesystem tools only, 5-minute timeout, 20 max turns) rewrites the memory store by reading all accumulated transcripts and existing memories.

7 Deterministic Behavior Guarantees

The hook system produces the following deterministic behaviors:

7.1 Invariant 1: No Tool Without Task

Hard Invariant

If `TodoManager.ByStatus(pending)` and `TodoManager.ByStatus(in_progress)` are both empty, **all non-exempt tools are blocked**. The agent *must* create a task before any tool execution.

Exception: Plan mode allows read-only tools. Task tools and plan mode tools are always exempt.

7.2 Invariant 2: Category-Based Tool Gating

Category Rules

- `planning` category ⇒ Write/Edit/Bash/Shell blocked (deterministic, no LLM)
- `debugging` category ⇒ only diagnostic tools allowed (deterministic, no LLM)
- Other categories ⇒ LLM relevance evaluation

7.3 Invariant 3: Priority-Ordered Execution

Hooks always execute in descending priority order. A block at priority 100 prevents all lower-priority hooks from running. This guarantees that security checks run before task enforcement, which runs before rate limiting.

7.4 Invariant 4: Guidance After Task Operations

After every `TaskCreate` or `TaskUpdate`, the `TaskGuidanceHook` deterministically injects the appropriate next-step guidance, preventing the agent from entering a confused state.

7.5 Invariant 5: Complete Event Capture

The `BronzeEventHook` (priority 5) captures *every* event, including blocked ones (which fire `EventToolBeforeExecu` before being blocked). This creates a complete audit trail with task correlation.

8 Complete Execution Trace

The following diagram shows the complete lifecycle of a single tool call, from agent request through all hook phases to final result:

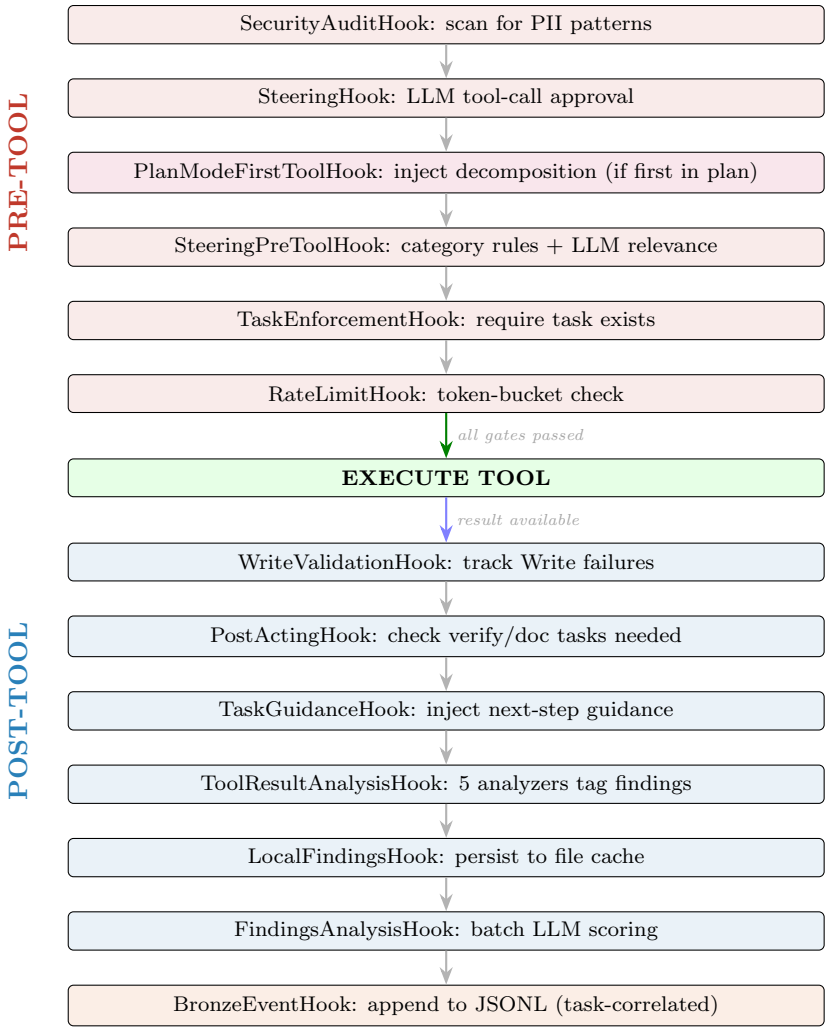

Figure 10: Complete hook chain for a single tool execution. Each box is one hook invocation. Any pre-tool hook can abort the entire chain.

9 Dependency Graph

The following diagram shows the runtime dependencies between subsystems:

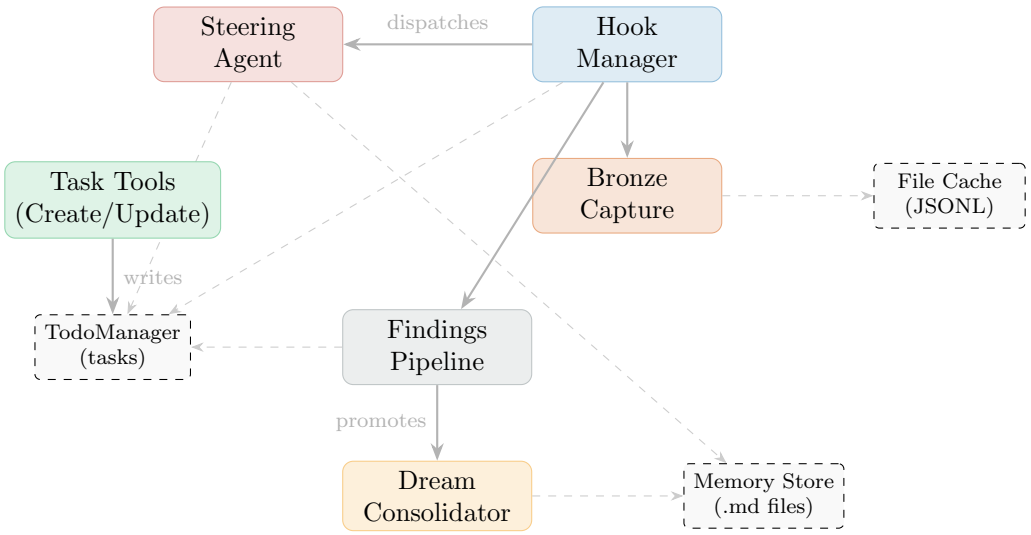

Figure 11: Runtime dependency graph. Solid arrows show control flow; dashed arrows show data reads. The TodoManager is the central shared state.

10 Summary

The Swarm SDK hook architecture enforces a disciplined agent workflow through layered, composable interceptors. The key design decisions are:

1. **Hooks run before validation.** Steering messages reach the agent before input validation errors, providing actionable guidance.
2. **Tasks are mandatory.** No tool execution without a tracked task. This creates full visibility and prevents untracked work.
3. **Plan mode is research mode.** Read-only tools work without tasks; write tools require commitment through task creation.
4. **Categories enforce discipline.** Planning tasks cannot write code; debugging tasks cannot wander into implementation.
5. **Everything is captured.** Bronze captures all events with task correlation. Silver scores for value. Gold persists as long-term memory.
6. **Guidance is proactive.** Post-tool hooks inject next-step guidance rather than waiting for the agent to discover constraints.

The result is an agent that plans before acting, tracks all work, stays focused on the current task, and builds institutional memory from its experiences.